

Supplementary Material for “Adaptive Time-aware Task Planning with LLMs for Efficient Execution under Uncertainty”

Anonymous Author

I. PROMPT TEMPLATE AND JSON OUTPUT SCHEMA

Fig. 3 of the main paper illustrates the structure of the tDAG-construction prompt. This section provides the prompt template and output schema used to construct the initial tDAG. The prompt consists of four blocks: (i) subtask-decomposition guidelines, (ii) an environment context that enumerates available objects and permissible interactions, (iii) temporal constraint inference guidelines with few-shot examples, and (iv) a strict JSON output schema. The full prompt files are released with the anonymous repository.

A. Prompt Template

You are a household robot task planner for an AI2-THOR or physical robot environment. Given a natural-language instruction and an environment context, decompose the instruction into Tasks and Subtasks.

[ROBOT CAPABILITIES]

Use only the following primitive actions:

GRASP, PLACE, OPEN, CLOSE, SLICE, NAVIGATE_TO, WAIT,

TOGGLE_ON, TOGGLE_OFF, MONITOR.

[ENVIRONMENT CONTEXT]

The environment context lists available object instances and the primitive actions permitted for each object. Use only objects and object-action pairs that appear in this context.

[SUBTASK DECOMPOSITION GUIDELINES]

1. Decompose each instruction into independently meaningful subtasks.
2. Each subtask must be executable as a sequence of primitive actions.
3. Do not invent objects, locations, or primitive actions.
4. Use exact object names from the environment context.
5. Insert temporal constraints only when the relation is necessary for task success or explicitly implied by the instruction.

[TEMPORAL CONSTRAINT INFERENCE]

Classify temporal relations as:

- Simple precedence: one subtask should occur before another, but no strict time-critical interval is required.
- Time-critical interval: the successor subtask must start after a specified interval and timing errors may cause task failure, e.g., overcooking, undercooking, overflow, or delayed unloading.

[OUTPUT FORMAT]

Return only a JSON array that follows the schema below.

B. JSON Output Schema

```
[
  {
    "Task": "<main task name>",
    "Subtasks": [
      {
        "Name": "<subtask name>",
        "Repetition": <integer>,
        "Type": "Interaction",
        "Executions": {
```

```

"Objects": {
  "<object name from environment context>": <integer>
},
"PrimitiveActions": [
  "<primitive action>",
  "...
]
},
"Duration": {
  "Type": "Controllable",
  "Interval": <integer>
},
"TemporalConstraints": [
{
  "Type": "<Before | After>",
  "Subtask": "<related subtask name>",
  "Interval": <integer>,
  "Urgency": <true | false>
}
]
}
]
}
]

```

The field `TemporalConstraints.Interval` specifies the interval Δ_{ij} associated with an edge from T_i to T_j . When `Urgency` is true, the edge is treated as time-critical and $t_i^{\text{end}} + \Delta_{ij}$ becomes the target start time of T_j . Otherwise, the interval specifies the earliest allowable start time. In the reported experiments, a time-critical constraint is considered satisfied when T_j starts within 12.5s of its target. This tolerance is used for evaluation and does not alter the interval encoded by the tDAG.

II. HYPERPARAMETER SETTINGS

Table I lists the key hyperparameters used in all reported experiments.

TABLE I: Key hyperparameter settings used in the reported experiments.

Parameter	Value	Role
W, D	10, 10	beam width / lookahead depth
η	0.1	risk tolerance for monitoring trigger
α_{sim}	0.001	observation-noise coefficient (simulation)
α_{vlm}	0.08	observation-noise coefficient (real-world VLM)
σ_{floor}^2	$(10\text{ s})^2$	VLM observation variance floor
ϵ_{min}^2	10^{-6} s^2	minimum simulation observation variance
Δ_0	$\{60, 80, 100, 120, 140\}\text{ s}$	initial prior mean
σ_0^2	$(40\text{ s})^2$	initial belief variance
Δ_{GT}	100 s	ground-truth interval for evaluation
TCSR tolerance	12.5 s	temporal-constraint evaluation tolerance
LLM	GPT-4o, temperature 0	tDAG construction
VLM	GPT-4.1, temperature 0, 14 few-shot anchors	real-world progress estimation

We use environment-specific observation models under the Gaussian likelihood $o_k \mid \Delta \sim \mathcal{N}(\Delta, \epsilon_k^2)$. Let $e_k = t_k - t_i^{\text{end}}$ denote the elapsed time since predecessor T_i completed. In simulation, the duration observation is sampled as

$$o_k \mid \Delta_{\text{GT}} \sim \mathcal{N}(\Delta_{\text{GT}}, \epsilon_k^2), \quad \epsilon_k^2 = \max\left\{\epsilon_{\text{min}}^2, \alpha_{\text{sim}}(\hat{\Delta}_{k-1} - e_k)^2\right\}.$$

Equivalently, defining $r_k = e_k / \hat{\Delta}_{k-1}$ shows that the progress-dependent variance contains the quadratic factor $(1 - r_k)^2$ described in the main paper.

For real-world monitoring, the VLM returns a quantized progress score $s_k \in \{0, 10, \dots, 130\}$, which is normalized and bounded as $r_k = \text{clip}(s_k/100, 0.05, 1.3)$; values above one represent estimated post-completion states. The duration observation is constructed from elapsed time and estimated progress as $o_k = e_k / r_k$. Its variance is modeled as

$$\epsilon_k^2 = \sigma_{\text{floor}}^2 + \alpha_{\text{vlm}}(1 - r_k)^2 o_k^2.$$

III. LLM-GENERATED TDAG CONSISTENCY

With temperature 0 and a fixed prompt/output schema, we measured the consistency of inferred time-critical relations across repeated translations. Count-level agreement with the intended task category held for 1,612/1,800 generated instruction–scene–translation outputs (89.6%). The 360 instruction–scene instances comprise 72 unique instructions instantiated in five scenes; translating each instance five times produces the 1,800 outputs. At the stricter edge level, the exact set of time-critical edges was identical across all five translations for 221/360 instruction–scene instances (61.4%), and appeared in at least four of five translations for 265/360 instances (73.6%). The rule-based refinement stage validates the schema, grounds object names, repairs basic executability violations such as missing NAVIGATE_TO or GRASP actions, and normalizes temporal annotations. Before refinement, 81.4% (1,465/1,800) of generated pairs satisfied the executor’s executability conditions; after applying the repair rules, all pairs did.

IV. DERIVATION OF THE POSTERIOR UPDATE

For completeness, we derive the closed-form posterior update used in the main paper. The posterior is proportional to the product of the Gaussian likelihood and the Gaussian prior:

$$p(\Delta \mid o^k) \propto \exp\left(-\frac{1}{2} \frac{(\Delta - o_k)^2}{\epsilon_k^2}\right) \exp\left(-\frac{1}{2} \frac{(\Delta - \hat{\Delta}_{k-1})^2}{\sigma_{k-1}^2}\right).$$

Expanding the quadratic terms in the exponent gives

$$p(\Delta \mid o^k) \propto \exp\left(-\frac{1}{2} \left[\left(\frac{1}{\epsilon_k^2} + \frac{1}{\sigma_{k-1}^2} \right) \Delta^2 - 2 \left(\frac{o_k}{\epsilon_k^2} + \frac{\hat{\Delta}_{k-1}}{\sigma_{k-1}^2} \right) \Delta + \mathcal{C} \right] \right),$$

where $\mathcal{C}$ collects terms independent of Δ . From this quadratic form, the posterior variance is identified as

$$\sigma_k^2 = \left(\frac{1}{\epsilon_k^2} + \frac{1}{\sigma_{k-1}^2} \right)^{-1} = \frac{\epsilon_k^2 \sigma_{k-1}^2}{\epsilon_k^2 + \sigma_{k-1}^2},$$

and completing the square yields the posterior mean

$$\hat{\Delta}_k = \frac{\sigma_{k-1}^2 o_k + \epsilon_k^2 \hat{\Delta}_{k-1}}{\epsilon_k^2 + \sigma_{k-1}^2},$$

which matches the update equations stated in the main paper.

V. SEARCH-PARAMETER AND SCHEDULER-COMPUTATION ANALYSIS

We vary the shared search parameter B ($W=D=B$) and measure schedule quality and scheduler computation time (CT). CT denotes the cumulative runtime of all scheduler invocations during one complete instruction execution (Table II). The shallow setting $B=1$ serves as a greedy single-step baseline.

TABLE II: Scalability of the scheduler under the correct-prior setting ($\eta=0.1$). Gap denotes the makespan difference from the exhaustive oracle schedule computed with ground-truth interval durations and without monitoring; CT denotes the cumulative scheduler runtime per instruction execution. Entries are means over five instruction sets.

Method	Task	Gap (s) ($\downarrow$)			CT (s) ($\downarrow$)		
		$B=1$	$B=10$	$B=20$	$B=1$	$B=10$	$B=20$
Ours	T2C1	6.70	0.55	0.55	0.14	1.34	2.01
	T2C2	4.48	3.68	3.26	0.21	1.92	2.70
	T3C1	27.03	15.99	13.41	0.27	3.67	5.72
	T3C2	24.75	18.52	14.34	0.43	6.67	12.63
Ours w/o Mon.	T2C1	6.14	0.00	0.00	0.10	0.50	0.52
	T2C2	4.73	3.50	3.50	0.12	0.57	0.59
	T3C1	9.73	4.21	3.79	0.15	1.38	1.82
	T3C2	14.56	4.85	3.86	0.23	2.66	3.48

At $B \in \{10, 20\}$, every generated schedule satisfied all temporal constraints for both methods. At $B = 1$, Instr. Feas. was 98.9% for Ours on T3C2 and 100% for all other method–task configurations. Instr. Feas. columns are omitted from Table II for compactness. Thus, the search parameter B primarily trades schedule efficiency (Gap) against computation time, with only a minor feasibility degradation at the greedy setting. Robustness under incorrect priors is evaluated separately in Sec. VI and in the belief-mismatch experiments of the main paper. Averaged over the four task settings, increasing B from 1 to 10 reduces the

makespan gap by 6.1 s, while increasing B to 20 saves only an additional 1.8 s and increases the average cumulative scheduler runtime per instruction execution from 3.4 s to 5.8 s.

The Bayesian belief update is a closed-form $O(1)$ computation per observation. With beam width W , lookahead depth D , and maximum branching factor A , each replanning call expands $O(WDA)$ candidate nodes after initialization.

VI. RISK-TOLERANCE SENSITIVITY

Table III reports the full risk-tolerance sensitivity results, separated into under-, correct-, and over-estimated prior regimes.

TABLE III: Schedule-level sensitivity to the risk tolerance η , separated into under-, correct-, and over-estimated priors. Instr. Feas. denotes the percentage of generated schedules satisfying all temporal constraints of an instruction. MS is omitted when Instr. Feas. $< 5\%$. Values are mean $\pm$ standard deviation over five decomposed instruction sets.

Regime	η	Instr. Feas. (%) ($\uparrow$)	MS (s) ($\downarrow$)	Mon./Int. ($\downarrow$)
Under	0.01	98.3 $\pm$ 0.0	216.7 $\pm$ 0.6	1.95 $\pm$ 0.00
	0.1	99.7 $\pm$ 0.0	213.7 $\pm$ 0.5	1.95 $\pm$ 0.00
	0.9	0.3 $\pm$ 0.2	–	0.00 $\pm$ 0.00
Correct	0.01	100.0 $\pm$ 0.0	210.8 $\pm$ 0.7	1.98 $\pm$ 0.00
	0.1	100.0 $\pm$ 0.0	181.6 $\pm$ 0.3	1.93 $\pm$ 0.00
	0.9	100.0 $\pm$ 0.0	175.1 $\pm$ 0.4	0.00 $\pm$ 0.00
Over	0.01	100.0 $\pm$ 0.0	181.3 $\pm$ 0.4	1.97 $\pm$ 0.00
	0.1	98.1 $\pm$ 0.1	179.4 $\pm$ 0.3	1.06 $\pm$ 0.00
	0.9	0.3 $\pm$ 0.2	–	0.00 $\pm$ 0.00

The regime-specific MS values in this table are schedule-level averages for the corresponding prior settings. The aggregated Incorrect MS in the main paper is computed over temporally feasible schedules; consequently, its 198.5 s value need not equal the arithmetic mean of the rounded Under and Over entries reported here. The setting $\eta = 0.9$ lies outside the valid range $\eta < 0.5$ required for a positive safety margin and is included as a stress test rather than as a sensitivity point. Its trigger falls after the expected transition, monitoring is never executed (Mon./Int. = 0), and instruction-level temporal feasibility collapses under incorrect priors.

VII. BELIEF UPDATES UNDER NON-GAUSSIAN DURATION FAMILIES

To examine robustness when the true duration distribution deviates from the Gaussian belief model, we compare the closed-form Gaussian-Bayesian update against a particle-filter (PF) belief update under three ground-truth duration families (Table IV).

TABLE IV: Particle filter vs. Gaussian-Bayesian updates under different ground-truth duration families ($W=D=10$, $\eta=0.1$, correct prior mean). Unlike the fixed-GT analysis in Table II, ground-truth durations are sampled from each family. Instr. Feas. denotes the percentage of instructions whose schedules satisfy all temporal constraints.

GT Family	Instr. Feas. (%) ($\uparrow$)		Gap (s) ($\downarrow$)		CT (s) ($\downarrow$)	
	Bayes	PF	Bayes	PF	Bayes	PF
Gaussian	99.31	99.08	23.64	22.05	3.45	3.53
Log-normal	85.71	86.92	27.85	25.42	3.54	3.37
Mixture	94.53	94.90	84.71	86.08	3.59	3.58

The Gaussian update remains competitive under log-normal and multimodal ground truths. Absolute performance still degrades under stronger distribution mismatch, so this result should be interpreted as evidence that the closed-form Gaussian update is a practical approximation rather than as a distribution-free guarantee.

VIII. EVALUATION-TOLERANCE SENSITIVITY

Table V reports TCSR re-scored under different evaluation tolerances.

TABLE V: TCSR (%) under different evaluation tolerances, computed by post-hoc re-scoring of saved execution logs.

Tolerance	Ours	w/o Mon.	EDF	CPM	Prog.	CaP
5.0 s	87.5	89.4	90.4	89.5	52.7	58.1
8.0 s	90.5	90.8	90.7	89.5	63.4	63.3
12.5 s	90.5	90.8	90.8	89.5	63.4	63.3

Stricter tolerances reduce tDAG-based schedulers by at most three percentage points, whereas LLM-only baselines drop by 5–11 points. This post-hoc check is not used as the main success metric; it only verifies that the relative trends are not driven by a single tolerance value.

IX. COMPARISON WITH AN ORACLE SCHEDULER

The oracle uses ground-truth interval durations and exhaustive depth-first search without monitoring. It provides a schedule-level lower bound on makespan but does not scale to large task graphs. Table VI reports the corresponding per-task makespan gaps. These values exclude execution-level stochasticity and wall-clock planning latency and are therefore not directly comparable to the execution-level makespans in the main paper.

TABLE VI: Schedule-level makespan comparison with the oracle. Gap is $MS_{\text{method}} - MS_{\text{oracle}}$.

Task	Oracle MS (s)	Ours Gap (s)	w/o Mon. Gap (s)
T2C1	156.4	+0.6	+0.0
T2C2	193.8	+3.7	+3.5
T3C1	156.8	+16.0	+4.2
T3C2	180.8	+18.5	+4.9

Under the correct-prior setting, the larger gaps for Ours reflect the scheduling overhead of monitoring; its robustness benefit is evaluated under belief mismatch in the main paper.

X. REAL-WORLD PERCEPTION DETAILS

The perception module estimates task progress from a single RGB frame using GPT-4.1 (temperature 0), prompted with 14 few-shot anchor frames per task (progress scores 0, 10, ..., 130, corresponding to progress ratios 0.0, 0.1, ..., 1.3). The prompts use task-specific brightness and hue rubrics: bright pink to dim red-pink for sausage, pale pink to dim deep red for tomato sauce, and bright white-cyan to dim teal for tea. The VLM returns a quantized score in the same anchor set, which is converted to a progress ratio and duration observation as described in Sec. II. Fig. 1 shows representative frames of the tomato-sauce task at three progress stages.

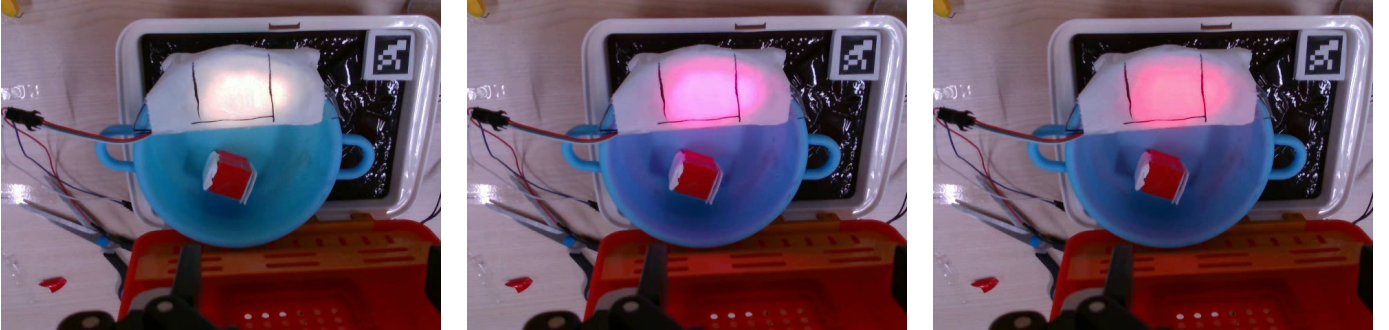


Fig. 1: Visual progress of the tomato-sauce task as observed by the perception module: (left) early pink-white glow (progress score 10, ratio 0.1); (middle) a more saturated reddish-pink glow (progress score 50, ratio 0.5); and (right) a dimmer red/coral-red glow (progress score 100, ratio 1.0).

XI. QUALITATIVE SCHEDULING EXAMPLES AND ERROR ANALYSIS

Fig. 2 contrasts the schedules generated by tDAG+CPM and our method for the same instruction under a mismatched prior, and Fig. 3 decomposes the real-world failure modes.

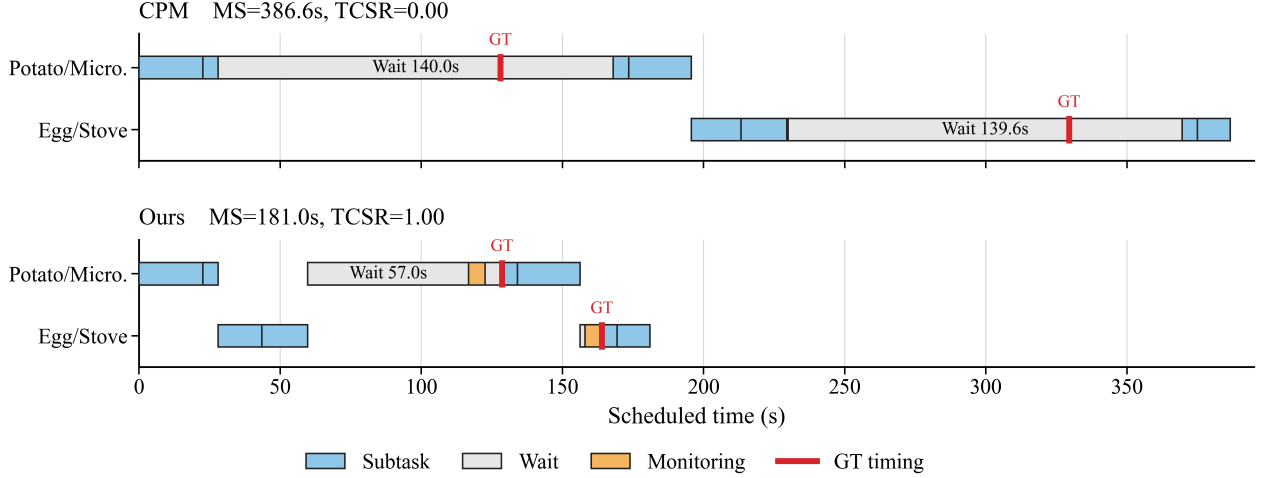


Fig. 2: Representative Gantt-chart comparison for the same T2C2 instruction under a mismatched temporal prior ($\hat{\Delta}_0=140$ s, $\Delta_{GT}=100$ s). The non-interleaved tDAG+CPM schedule follows the overestimated prior and misses the time-critical timings, whereas our method inserts monitoring, refines the interval belief online, and interleaves subtasks, yielding a shorter makespan.

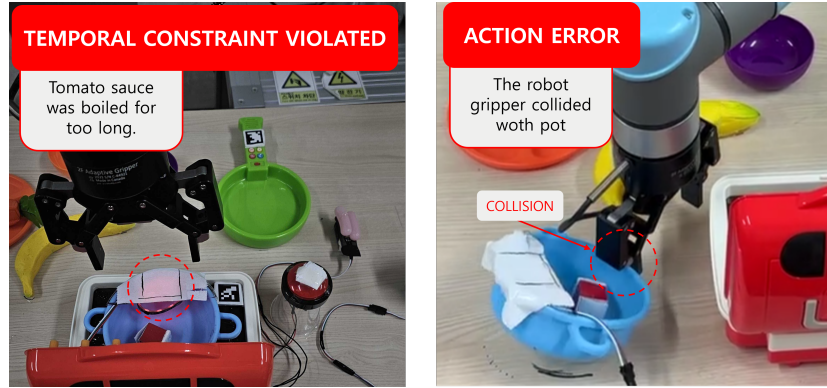


Fig. 3: Failure-mode decomposition in the real-world experiments: temporal-constraint violations versus low-level action errors across prior settings. Baselines without online belief refinement fail predominantly through temporal violations under mismatched priors, whereas the residual failures of our method stem mainly from low-level perception and manipulation noise rather than the scheduler.

XII. SIMULATION INSTRUCTION LIST

Table VII lists the simulation instructions used in AI2-THOR. The dataset contains 72 unique instructions: 18 instructions for each of the four task/constraint cases. Each instruction is instantiated across five AI2-THOR scenes and evaluated over five repeated trials.

TABLE VII: The 72 unique natural-language instructions used in simulation.

Case	ID	Instruction
T2C1	1	boil potato and put apple and lettuce in fridge
T2C1	2	boil water with pot and put apple and lettuce in fridge
T2C1	3	cook egg and put apple and lettuce in fridge
T2C1	4	cook egg and wash a butterknife
T2C1	5	cook egg and wash a spatula
T2C1	6	fill bowl with water and put apple and lettuce in fridge
T2C1	7	heat the bread using microwave and wash a spatula
T2C1	8	heat the bread using microwave and wash all fork and spoon
T2C1	9	heat the bread using microwave and wash apple and lettuce
T2C1	10	heat the bread using microwave and wash plate and cup
T2C1	11	heat the potato using microwave and wash a tomato
T2C1	12	heat the potato using microwave and wash all fork and spoon

Case	ID	Instruction
T2C1	13	heat the potato using microwave and wash apple and lettuce
T2C1	14	heat the potato using microwave and wash plate and cup
T2C1	15	make a coffee and put apple and lettuce in fridge
T2C1	16	make a coffee and wash a spatula
T2C1	17	make a coffee and wash a tomato
T2C1	18	make a coffee and wash apple and lettuce
T2C2	1	boil potato and heat the bread using microwave
T2C2	2	boil potato and make a coffee
T2C2	3	boil water with pot and heat the bread using microwave
T2C2	4	boil water with pot and heat the potato using microwave
T2C2	5	boil water with pot and make a coffee
T2C2	6	fill bowl with water and cook egg
T2C2	7	fill bowl with water and heat the bread using microwave
T2C2	8	fill bowl with water and heat the potato using microwave
T2C2	9	fill bowl with water and make a coffee
T2C2	10	fill pot with water and cook egg
T2C2	11	fill pot with water and heat the bread using microwave
T2C2	12	fill pot with water and heat the potato using microwave
T2C2	13	fill pot with water and make a coffee
T2C2	14	heat the bread using microwave and cook egg
T2C2	15	heat the bread using microwave and make a coffee
T2C2	16	heat the potato using microwave and cook egg
T2C2	17	heat the potato using microwave and make a coffee
T2C2	18	make a coffee and cook egg
T3C1	1	cook egg and wash a butterknife and wash apple and lettuce
T3C1	2	cook egg and wash a spatula and wash apple and lettuce
T3C1	3	cook egg and wash a tomato and wash all fork and spoon
T3C1	4	cook egg and wash all fork and spoon and wash apple and lettuce
T3C1	5	heat the bread using microwave and put apple and lettuce in fridge and wash all fork and spoon
T3C1	6	heat the bread using microwave and wash a butterknife and wash all fork and spoon
T3C1	7	heat the bread using microwave and wash a butterknife and wash plate and cup
T3C1	8	heat the bread using microwave and wash a spatula and wash plate and cup
T3C1	9	heat the bread using microwave and wash a tomato and wash a spatula
T3C1	10	heat the bread using microwave and wash a tomato and wash all fork and spoon
T3C1	11	heat the potato using microwave and put apple and lettuce in fridge and wash a butterknife
T3C1	12	heat the potato using microwave and put apple and lettuce in fridge and wash a spatula
T3C1	13	heat the potato using microwave and put apple and lettuce in fridge and wash a tomato
T3C1	14	heat the potato using microwave and wash a butterknife and wash a spatula
T3C1	15	heat the potato using microwave and wash a spatula and wash plate and cup
T3C1	16	heat the potato using microwave and wash a tomato and wash plate and cup
T3C1	17	make a coffee and wash a butterknife and wash apple and lettuce
T3C1	18	make a coffee and wash a spatula and wash apple and lettuce
T3C2	1	boil potato and heat the bread using microwave and put apple and lettuce in fridge
T3C2	2	boil potato and make a coffee and put apple and lettuce in fridge
T3C2	3	boil water with pot and make a coffee and put apple and lettuce in fridge
T3C2	4	fill bowl with water and cook egg and put apple and lettuce in fridge
T3C2	5	fill bowl with water and heat the bread using microwave and put apple and lettuce in fridge
T3C2	6	fill bowl with water and make a coffee and put apple and lettuce in fridge
T3C2	7	fill pot with water and cook egg and put apple and lettuce in fridge
T3C2	8	heat the bread using microwave and cook egg and wash plate and cup
T3C2	9	heat the bread using microwave and make a coffee and wash all fork and spoon
T3C2	10	heat the potato using microwave and cook egg and wash a spatula
T3C2	11	heat the potato using microwave and cook egg and wash a tomato
T3C2	12	heat the potato using microwave and cook egg and wash all fork and spoon
T3C2	13	heat the potato using microwave and cook egg and wash plate and cup
T3C2	14	heat the potato using microwave and make a coffee and wash a butterknife
T3C2	15	heat the potato using microwave and make a coffee and wash a tomato
T3C2	16	heat the potato using microwave and make a coffee and wash all fork and spoon
T3C2	17	heat the potato using microwave and make a coffee and wash apple and lettuce
T3C2	18	make a coffee and cook egg and wash a spatula

XIII. REAL-WORLD INSTRUCTION LIST

Table VIII lists all 21 instructions used in the real-world evaluation.

TABLE VIII: The 21 natural-language instructions used in the real-world evaluation (7 per case).

Case	ID	Instruction
T2C1	1	cook sausage and place banana on plate
T2C1	2	cook sausage and place carrot on plate
T2C1	3	cook sausage and put green cup in sink
T2C1	4	make tomato sauce and place banana on plate
T2C1	5	make tomato sauce and place carrot on plate
T2C1	6	make tea in teapot and place banana on plate
T2C1	7	make tea in teapot and place carrot on plate
T3C1	1	cook sausage and put green cup in sink and put orange bowl in sink
T3C1	2	cook sausage and put orange bowl in sink and place carrot on plate
T3C1	3	cook sausage and put purple bowl in sink and place banana on plate
T3C1	4	make tomato sauce and place banana on plate and put orange bowl in sink
T3C1	5	make tomato sauce and put green cup in sink and place carrot on plate
T3C1	6	make tea in teapot and place banana on plate and put orange bowl in sink
T3C1	7	make tea in teapot and put green cup in sink and place banana on plate
T3C2	1	cook sausage and make tea in teapot and place banana on plate
T3C2	2	cook sausage and make tea in teapot and place carrot on plate
T3C2	3	cook sausage and make tea in teapot and put green cup in sink
T3C2	4	cook sausage and make tea in teapot and put orange bowl in sink
T3C2	5	make tomato sauce and make tea in teapot and place banana on plate
T3C2	6	make tomato sauce and make tea in teapot and place carrot on plate
T3C2	7	make tomato sauce and make tea in teapot and put green cup in sink